Survival Analysis in R

FAROUK OSMAN

2026-09-25

Introduction

This markdown file are personal notes that follow the Survival Analysis in R tutorial by Emily C. Zabor https://www.emilyzabor.com/survival-analysis-in-r.html (https://www.emilyzabor.com/survival-analysis-in-r.html)

Load packages

```
library(knitr)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(survival)
library(ggplot2)
library(tibble)
```

Part 1; The lung dataset

Load packages

```
library(lubridate)
```

```
##
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```
library(ggsurvfit)
library(gtsummary)
library(tidycmprsk)
```

```
##
## Attaching package: 'tidycmprsk'
```

```
## The following object is masked from 'package:gtsummary':
##
##     trial
```

```
library(condSURV)
```

The data contain subjects with advanced lung cancer from the North Central Cancer Treatment Group

Variables to focus on; - time; Observed survival time in days - status 1=censored, 2=dead - sex; 1=Male, 2=Female

First we recode status to a standard 1 0 formart

```
data(cancer, package = "survival")

lung <-
  lung |>
  mutate(
    status = recode(status, `1`= 0, `2` = 1)
  )
```

Inspect First six observations

```
head(lung[c("time", "status", "sex")])
```

```
##   time status sex
## 1  306      1   1
## 2  455      1   1
## 3 1010      0   1
## 4  210      1   1
## 5  883      1   1
## 6 1022      0   1
```

NOTE; When using Surv() ; make sure the event column is properly formated to ; true/fales or 1/0 or 2/1 ; Otherwise you wont know if its wrong

Calculating Surival times

Data will usually show up as start and end dates; rather than pre-calculated survival times. Create a small data frame (date_ex) with dates and convert it to survival times - sx_date ; surgery date - last_fup_date; last follow up date

```
date_ex <-
  tibble(
    sx_date = c("2007-06-22", "2004-02-13", "2010-10-27"),
    last_fup_date = c("2017-04-15","2018-07-04", "2016-10-31")
  )
View(date_ex)
```

Using the lubridate package and ymd function; we convert date_ex from characters to dates

```
date_ex <- date_ex |>
  mutate(
    sx_date = ymd(sx_date),
    last_fup_date = ymd(last_fup_date)
  )

#check if converted from character to date
class(date_ex$sx_date)
```

```
## [1] "Date"
```

```
class(date_ex$last_fup_date)
```

```
## [1] "Date"
```

Now that the dates are formatted, we need to calculate the difference between start and end dates in some units, usually months or years. Using the {lubridate} package, the operator %–% designates a time interval, which is then converted to the number of elapsed seconds using as.duration() and finally converted to years by dividing by dyears(1), which gives the number of seconds in a year.

```
date_ex <- date_ex |>
  mutate(
    os_years = as.duration(sx_date %--% last_fup_date) / dyears(1)
  )

#"Take the date_ex table, add a new column called os_yrs that contains the time in
years between the surgery date and the last follow-up date, save it back into date_
ex, then show me the table." / dyears(1) simply means "Divide by the number of seco
nds in one year." dyears(1) gives R the exact number of seconds in a year, so divid
ing converts your duration from seconds into years.
```

Now we have a column for survival times in years- Back to the Lung dataset

Creating survival objects and curves

Creating a Surv() object is the same as declaring survival data in stata Done as Surv(time, status)

Lets create a surv() object of the first 10 rows

```
Surv(lung$time, lung$status)[1:10]
```

```
##  [1]  306   455  1010+  210   883  1022+  310   361   218   166
```

Create an object s1 and use the survfit() function ; this actually estimates the survival curve using the Kaplan-Meier method and computes all the survival probabilities at each time point, then look at the internal structure using str(). This basically produces a KM table

```
s1 <- survfit(Surv(time, status) ~ 1, data = lung)
str(s1)
```

```
## List of 17
##  $ n        : int 228
##  $ time     : num [1:186] 5 11 12 13 15 26 30 31 53 54 ...
##  $ n.risk   : num [1:186] 228 227 224 223 221 220 219 218 217 215 ...
##  $ n.event  : num [1:186] 1 3 1 2 1 1 1 1 2 1 ...
##  $ n.censor : num [1:186] 0 0 0 0 0 0 0 0 0 0 ...
##  $ surv     : num [1:186] 0.996 0.982 0.978 0.969 0.965 ...
##  $ std.err  : num [1:186] 0.0044 0.00885 0.00992 0.01179 0.01263 ...
##  $ cumhaz   : num [1:186] 0.00439 0.0176 0.02207 0.03103 0.03556 ...
##  $ std.chaz : num [1:186] 0.00439 0.0088 0.00987 0.01173 0.01257 ...
##  $ type     : chr "right"
##  $ logse    : logi TRUE
##  $ conf.int : num 0.95
##  $ conf.type: chr "log"
##  $ lower    : num [1:186] 0.987 0.966 0.959 0.947 0.941 ...
##  $ upper    : num [1:186] 1 1 0.997 0.992 0.989 ...
##  $ t0       : num 0
##  $ call     : language survfit(formula = Surv(time, status) ~ 1, data = lung)
##  - attr(*, "class")= chr "survfit"
```

```
summary(s1)
```

```
## Call: survfit(formula = Surv(time, status) ~ 1, data = lung)
##
##  time n.risk n.event survival std.err lower 95% CI upper 95% CI
##     5    228       1   0.9956 0.00438       0.9871        1.000
##    11    227       3   0.9825 0.00869       0.9656        1.000
##    12    224       1   0.9781 0.00970       0.9592        0.997
##    13    223       2   0.9693 0.01142       0.9472        0.992
##    15    221       1   0.9649 0.01219       0.9413        0.989
##    26    220       1   0.9605 0.01290       0.9356        0.986
##    30    219       1   0.9561 0.01356       0.9299        0.983
##    31    218       1   0.9518 0.01419       0.9243        0.980
##    53    217       2   0.9430 0.01536       0.9134        0.974
##    54    215       1   0.9386 0.01590       0.9079        0.970
##    59    214       1   0.9342 0.01642       0.9026        0.967
##    60    213       2   0.9254 0.01740       0.8920        0.960
##    61    211       1   0.9211 0.01786       0.8867        0.957
##    62    210       1   0.9167 0.01830       0.8815        0.953
##    65    209       2   0.9079 0.01915       0.8711        0.946
##    71    207       1   0.9035 0.01955       0.8660        0.943
##    79    206       1   0.8991 0.01995       0.8609        0.939
##    81    205       2   0.8904 0.02069       0.8507        0.932
##    88    203       2   0.8816 0.02140       0.8406        0.925
##    92    201       1   0.8772 0.02174       0.8356        0.921
##    93    199       1   0.8728 0.02207       0.8306        0.917
##    95    198       2   0.8640 0.02271       0.8206        0.910
##   105    196       1   0.8596 0.02302       0.8156        0.906
##   107    194       2   0.8507 0.02362       0.8056        0.898
##   110    192       1   0.8463 0.02391       0.8007        0.894
##   116    191       1   0.8418 0.02419       0.7957        0.891
##   118    190       1   0.8374 0.02446       0.7908        0.887
##   122    189       1   0.8330 0.02473       0.7859        0.883
##   131    188       1   0.8285 0.02500       0.7810        0.879
##   132    187       2   0.8197 0.02550       0.7712        0.871
##   135    185       1   0.8153 0.02575       0.7663        0.867
##   142    184       1   0.8108 0.02598       0.7615        0.863
##   144    183       1   0.8064 0.02622       0.7566        0.859
##   145    182       2   0.7975 0.02667       0.7469        0.852
##   147    180       1   0.7931 0.02688       0.7421        0.848
##   153    179       1   0.7887 0.02710       0.7373        0.844
##   156    178       2   0.7798 0.02751       0.7277        0.836
##   163    176       3   0.7665 0.02809       0.7134        0.824
##   166    173       2   0.7577 0.02845       0.7039        0.816
##   167    171       1   0.7532 0.02863       0.6991        0.811
##   170    170       1   0.7488 0.02880       0.6944        0.807
##   175    167       1   0.7443 0.02898       0.6896        0.803
##   176    165       1   0.7398 0.02915       0.6848        0.799
##   177    164       1   0.7353 0.02932       0.6800        0.795
##   179    162       2   0.7262 0.02965       0.6704        0.787
##   180    160       1   0.7217 0.02981       0.6655        0.783
##   181    159       2   0.7126 0.03012       0.6559        0.774
##   182    157       1   0.7081 0.03027       0.6511        0.770
##   183    156       1   0.7035 0.03041       0.6464        0.766
##   186    154       1   0.6989 0.03056       0.6416        0.761
##   189    152       1   0.6943 0.03070       0.6367        0.757
```

```
##   194    149       1   0.6897 0.03085       0.6318        0.753
##   197    147       1   0.6850 0.03099       0.6269        0.749
##   199    145       1   0.6803 0.03113       0.6219        0.744
##   201    144       2   0.6708 0.03141       0.6120        0.735
##   202    142       1   0.6661 0.03154       0.6071        0.731
##   207    139       1   0.6613 0.03168       0.6020        0.726
##   208    138       1   0.6565 0.03181       0.5970        0.722
##   210    137       1   0.6517 0.03194       0.5920        0.717
##   212    135       1   0.6469 0.03206       0.5870        0.713
##   218    134       1   0.6421 0.03218       0.5820        0.708
##   222    132       1   0.6372 0.03231       0.5769        0.704
##   223    130       1   0.6323 0.03243       0.5718        0.699
##   226    126       1   0.6273 0.03256       0.5666        0.694
##   229    125       1   0.6223 0.03268       0.5614        0.690
##   230    124       1   0.6172 0.03280       0.5562        0.685
##   239    121       2   0.6070 0.03304       0.5456        0.675
##   245    117       1   0.6019 0.03316       0.5402        0.670
##   246    116       1   0.5967 0.03328       0.5349        0.666
##   267    112       1   0.5913 0.03341       0.5294        0.661
##   268    111       1   0.5860 0.03353       0.5239        0.656
##   269    110       1   0.5807 0.03364       0.5184        0.651
##   270    108       1   0.5753 0.03376       0.5128        0.645
##   283    104       1   0.5698 0.03388       0.5071        0.640
##   284    103       1   0.5642 0.03400       0.5014        0.635
##   285    101       2   0.5531 0.03424       0.4899        0.624
##   286     99       1   0.5475 0.03434       0.4841        0.619
##   288     98       1   0.5419 0.03444       0.4784        0.614
##   291     97       1   0.5363 0.03454       0.4727        0.608
##   293     94       1   0.5306 0.03464       0.4669        0.603
##   301     91       1   0.5248 0.03475       0.4609        0.597
##   303     89       1   0.5189 0.03485       0.4549        0.592
##   305     87       1   0.5129 0.03496       0.4488        0.586
##   306     86       1   0.5070 0.03506       0.4427        0.581
##   310     85       2   0.4950 0.03523       0.4306        0.569
##   320     82       1   0.4890 0.03532       0.4244        0.563
##   329     81       1   0.4830 0.03539       0.4183        0.558
##   337     79       1   0.4768 0.03547       0.4121        0.552
##   340     78       1   0.4707 0.03554       0.4060        0.546
##   345     77       1   0.4646 0.03560       0.3998        0.540
##   348     76       1   0.4585 0.03565       0.3937        0.534
##   350     75       1   0.4524 0.03569       0.3876        0.528
##   351     74       1   0.4463 0.03573       0.3815        0.522
##   353     73       2   0.4340 0.03578       0.3693        0.510
##   361     70       1   0.4278 0.03581       0.3631        0.504
##   363     69       2   0.4154 0.03583       0.3508        0.492
##   364     67       1   0.4092 0.03582       0.3447        0.486
##   371     65       2   0.3966 0.03581       0.3323        0.473
##   387     60       1   0.3900 0.03582       0.3258        0.467
##   390     59       1   0.3834 0.03582       0.3193        0.460
##   394     58       1   0.3768 0.03580       0.3128        0.454
##   426     55       1   0.3700 0.03580       0.3060        0.447
##   428     54       1   0.3631 0.03579       0.2993        0.440
##   429     53       1   0.3563 0.03576       0.2926        0.434
##   433     52       1   0.3494 0.03573       0.2860        0.427
```

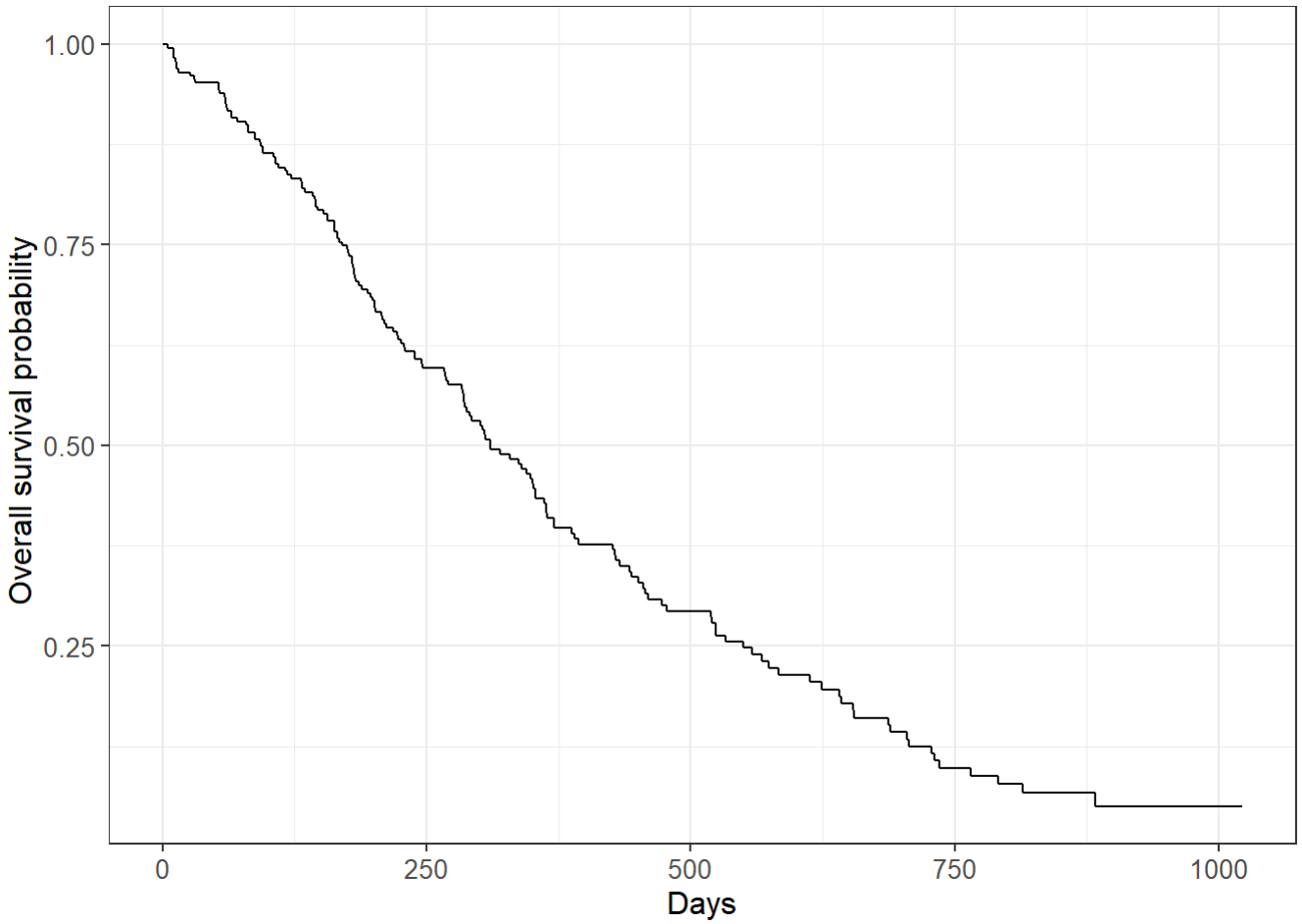
```
##   442     51       1   0.3426 0.03568       0.2793        0.420
##   444     50       1   0.3357 0.03561       0.2727        0.413
##   450     48       1   0.3287 0.03555       0.2659        0.406
##   455     47       1   0.3217 0.03548       0.2592        0.399
##   457     46       1   0.3147 0.03539       0.2525        0.392
##   460     44       1   0.3076 0.03530       0.2456        0.385
##   473     43       1   0.3004 0.03520       0.2388        0.378
##   477     42       1   0.2933 0.03508       0.2320        0.371
##   519     39       1   0.2857 0.03498       0.2248        0.363
##   520     38       1   0.2782 0.03485       0.2177        0.356
##   524     37       2   0.2632 0.03455       0.2035        0.340
##   533     34       1   0.2554 0.03439       0.1962        0.333
##   550     32       1   0.2475 0.03423       0.1887        0.325
##   558     30       1   0.2392 0.03407       0.1810        0.316
##   567     28       1   0.2307 0.03391       0.1729        0.308
##   574     27       1   0.2221 0.03371       0.1650        0.299
##   583     26       1   0.2136 0.03348       0.1571        0.290
##   613     24       1   0.2047 0.03325       0.1489        0.281
##   624     23       1   0.1958 0.03297       0.1407        0.272
##   641     22       1   0.1869 0.03265       0.1327        0.263
##   643     21       1   0.1780 0.03229       0.1247        0.254
##   654     20       1   0.1691 0.03188       0.1169        0.245
##   655     19       1   0.1602 0.03142       0.1091        0.235
##   687     18       1   0.1513 0.03090       0.1014        0.226
##   689     17       1   0.1424 0.03034       0.0938        0.216
##   705     16       1   0.1335 0.02972       0.0863        0.207
##   707     15       1   0.1246 0.02904       0.0789        0.197
##   728     14       1   0.1157 0.02830       0.0716        0.187
##   731     13       1   0.1068 0.02749       0.0645        0.177
##   735     12       1   0.0979 0.02660       0.0575        0.167
##   765     10       1   0.0881 0.02568       0.0498        0.156
##   791      9       1   0.0783 0.02462       0.0423        0.145
##   814      7       1   0.0671 0.02351       0.0338        0.133
##   883      4       1   0.0503 0.02285       0.0207        0.123
```

Key components; time: the timepoints at which the curve has a step, i.e. at least one event occurred surv: the estimate of survival at the corresponding time

Kaplan-Meier plots

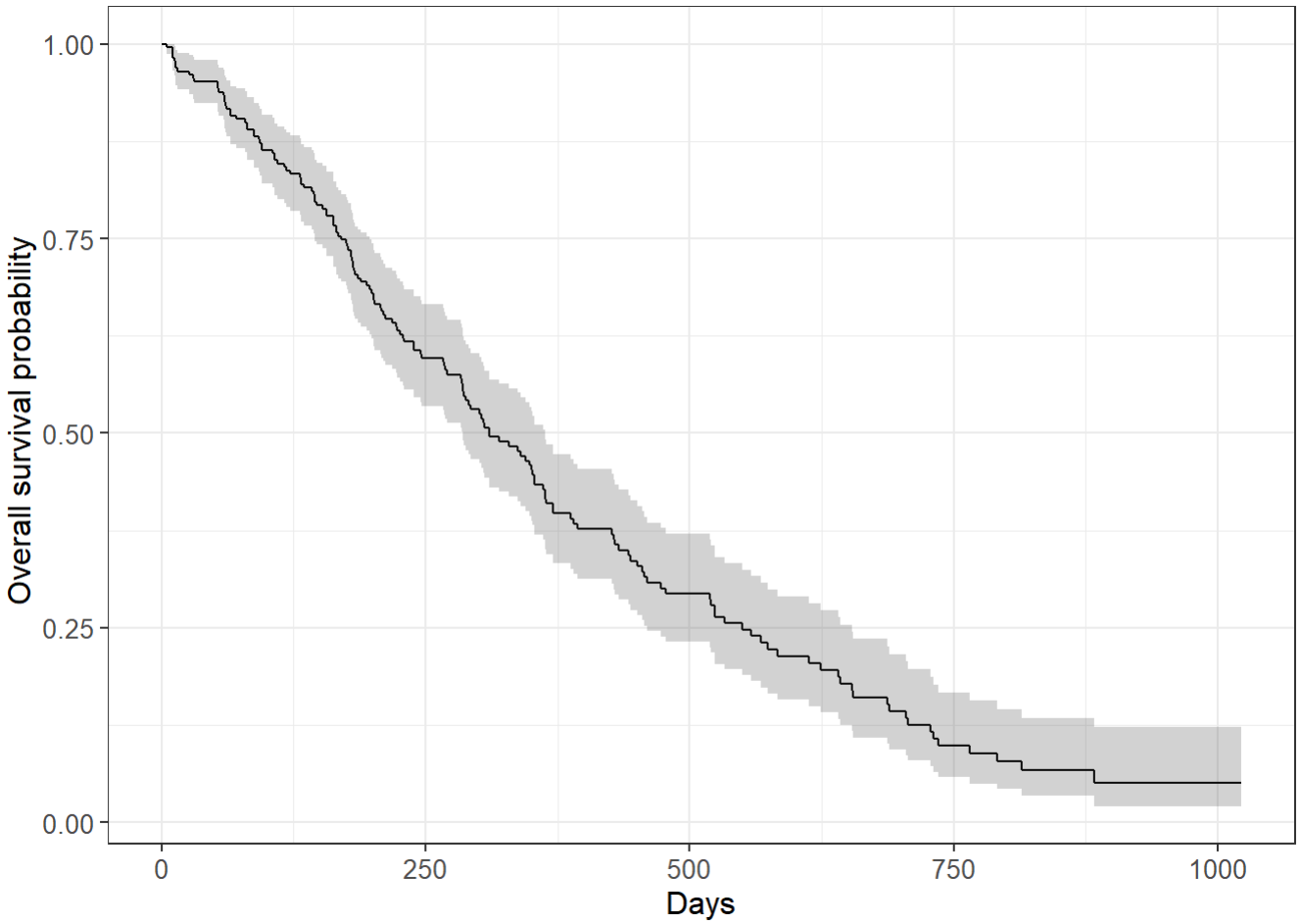
We use the ggsurvfit package to generate KM plots It uses the already created survfit object using the survfit2() function

```
survfit2(Surv(time, status) ~ 1, data = lung) |>
  ggsurvfit() +
  labs(
    x = "Days",
    y = " Overall survival probability"
  )
```



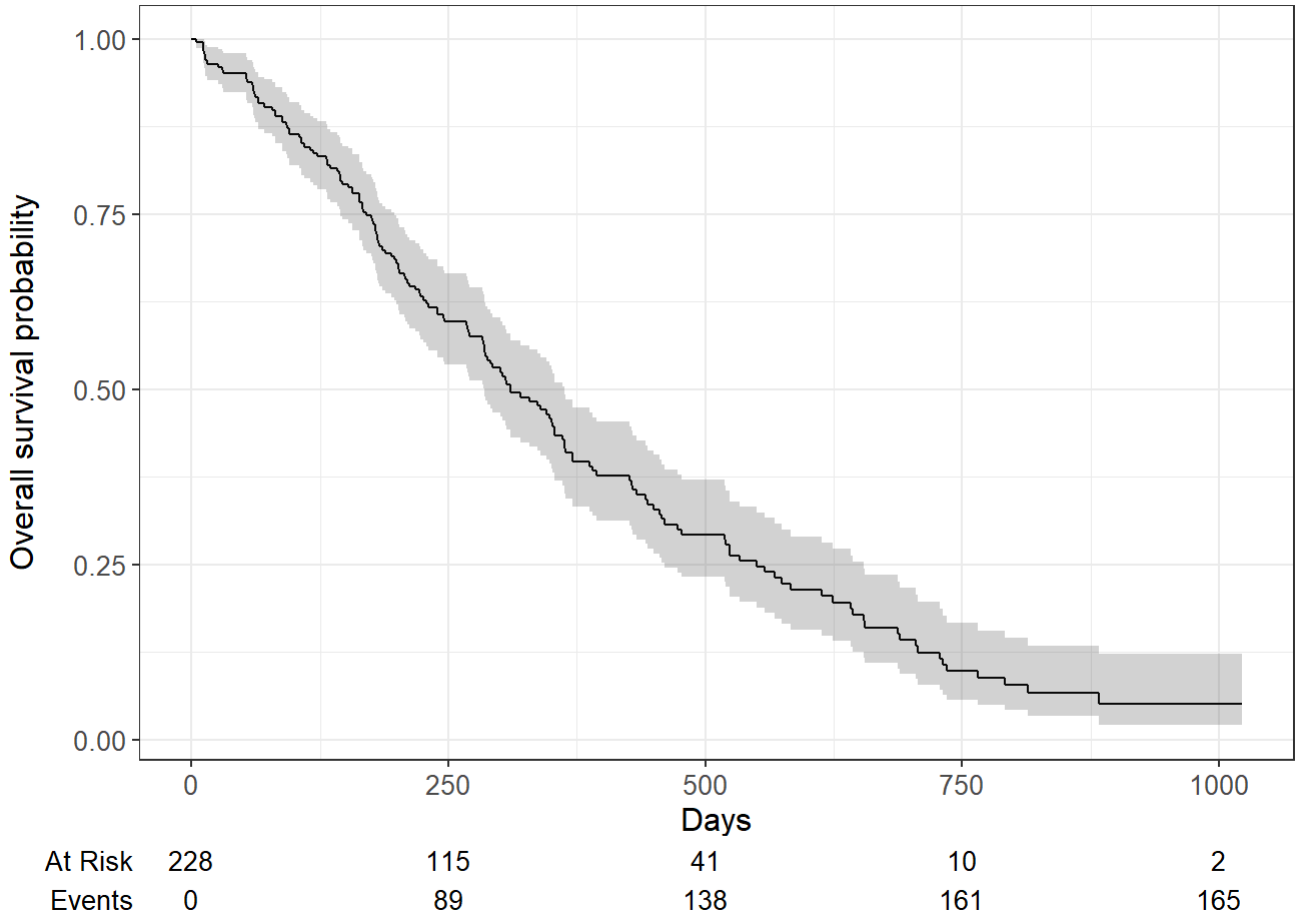
This only shows the step function; we can add confidence intervals to the plot

```
survfit2(Surv(time, status) ~ 1, data = lung) |>
  ggsurvfit() +
  labs(
    x = "Days",
    y = " Overall survival probability"
  ) +
  add_confidence_interval()
```



Typically we will also want to see the numbers at risk in a table below the x-axis. We can add this using add_risktable():

```
survfit2(Surv(time, status) ~ 1, data = lung) |>
  ggsurvfit() +
  labs(
    x = "Days",
    y = "Overall survival probability"
  ) +
  add_confidence_interval() +
  add_risktable()
```



Estimating x-year survival

We are extracting a single survival probability estimate at a specific time point from the KM curve, for example asking "what percentage of patients survived to 1 year?" and showing why the KM method gives a more accurate answer than a naive calculation that ignores censoring.

REMEMBER;

`survfit()` = Kaplan-Meier calculation

`Surv()` just labels your data. `survfit()` is what actually computes the KM estimate. Everything else — plots, tables, summary statistics — is just reading from what `survfit()` calculated.

For example, to estimate the probability of surviving to year, use summary with the times argument (Note: the time variable in the lung data is actually in days, so we need to use times = 365.25)

```
summary(survfit(Surv(time, status) ~ 1, data = lung), times = 365.25)
```

```
## Call: survfit(formula = Surv(time, status) ~ 1, data = lung)
##
##  time n.risk n.event survival std.err lower 95% CI upper 95% CI
##   365     65     121    0.409  0.0358        0.345        0.486
```

We find that the 1-year probability of survival in this study is 41%(0.409).

The associated lower and upper bounds of the 95% confidence interval are also displayed.

We can produce nice tables of x-time survival probability estimates using the tbl_survfit() function from the {gtsummary} package:

```
survfit(Surv(time, status) ~ 1, data = lung) |>
  tbl_survfit(
    times = 365.25,
    label_header = "**1-year survival  (95% CI)"
    )
```

Characteristic	**1-year survival (95% CI)
Overall	41% (34%, 49%)

Estimating median survival time

We can obtain the median survival directly from the survfit object:

```
survfit(Surv(time, status) ~ 1, data = lung)
```

```
## Call: survfit(formula = Surv(time, status) ~ 1, data = lung)
##
##        n events median 0.95LCL 0.95UCL
## [1,] 228    165    310     285     363
```

We see the median survival time is 310 days The lower and upper bounds of the 95% confidence interval are also displayed. We can display this in a table using tbl_survfit

```
survfit(Surv(time, status) ~ 1, data = lung) |>
  tbl_survfit(
    probs = 0.5,
    label_header ="**Median survival (95% CI)**"
  )
```

Characteristic	**Median survival (95% CI)**
Overall	310 (285, 363)

Comparing survival times between groups

This section compares survival times between two groups (males vs females) using the **log-rank test** (`survdiff()`), which is the survival analysis equivalent of a t-test or chi-square test. It tests whether the difference in survival between groups is statistically significant by comparing observed vs expected deaths in each group across the entire follow-up period, producing a p-value. It does not quantify the size of the difference — only whether one exists. For that, Cox regression is needed.

```
survdiff(Surv(time, status) ~ sex, data = lung)
```

```
## Call:
## survdiff(formula = Surv(time, status) ~ sex, data = lung)
##
##         N Observed Expected (O-E)^2/E (O-E)^2/V
## sex=1 138      112     91.6      4.55      10.3
## sex=2  90       53     73.4      5.68      10.3
##
##  Chisq= 10.3  on 1 degrees of freedom, p= 0.001
```

We see that there was a significant difference in overall survival according to sex in the lung data, with a p-value of p = 0.001.

The Cox regression model

Cox regression (`coxph()`) is used to quantify the effect of a covariate on survival as a **hazard ratio (HR)**, and can handle multiple covariates simultaneously unlike the log-rank test. A HR < 1 means reduced risk of death, HR > 1 means increased risk. Results are displayed using `tbl_regression(exp = TRUE)` which exponentiates the coefficients to give HRs instead of log HRs. In this section sex is the only covariate used, giving HR = 0.59 meaning females have a 41% lower hazard of death than males. Note that the proportional hazards assumption is not checked here ; that is covered in Part 4.

Lets run a cox model with only sex

```
coxph(Surv(time, status) ~ sex, data = lung)
```

```
## Call:
## coxph(formula = Surv(time, status) ~ sex, data = lung)
##
##        coef exp(coef) se(coef)      z       p
## sex -0.5310    0.5880   0.1672 -3.176 0.00149
##
## Likelihood ratio test=10.63  on 1 df, p=0.001111
## n= 228, number of events= 165
```

LOOK AT THE LR test (p=0.001111) ; This is saying the model with sex in it fits significantly better than an empty model ; confirming that sex is a meaningful predictor of survival.

We can put the above results in a table using tbl_regression; and exponentionate is set to TRUE; so that the table shows HR rather than log HR

```
coxph(Surv(time, status) ~sex, data = lung) |>
  tbl_regression(exponentiate = TRUE)
```

Characteristic	HR	95% CI	p-value
sex	0.59	0.42, 0.82	0.001

Abbreviations: CI = Confidence Interval, HR = Hazard Ratio

So the HR = 0.59 implies that 0.59 times as many females are dying as males, at any given time. Stated differently, females have a significantly lower hazard of death than males in these data. (Remember ; HR<1 indicates reduced hazard of death and vice versa)

Part 2: Landmark Analysis and Time Dependent Covariates